TP02 Report – Processing Large CSV Files

1.Introduction

في مجال Big Data

يتم التعامل مع ملفات بيانات ضخمة لا يمكن تحميلها بالكامل في الذاكرة.
في هذا العمل التطبيقي قمنا بدراسة طرق مختلفة لمعالجة ملفCSV

كبير باستخدام لغة **Python** ومكتبات تحليل البيانات .

تم استخدامDataset من منصةKaggleوهي:

IBM Transactions for Anti-Money Laundering (AML)

يحتوي dataset على معاملات مالية كبيرة تُستخدم لدراسة عمليات **مكافحة غسل الأموال**

.على معاملات مالية كبيرة تُستخدم لدراسة عمليات **مكافحة غسل الأموال**

الملف الذي تم استخدامه في التجربة هو:

HI-Large_Trans.csv

وهو ملف كبير يحتوي على ملايين السجلات.

هدف هذا المشروع هو مقارنة ثلاث طرق لمعالجة ملفات CSV الكبيرة

1. Pandas باستخدام**Chunk Size**
2. **Dask** استخدام مكتبة
3. **Compression** استخدام

2. Dataset Description

البيانات المستخدمة هي معاملات مالية تحتوي على معلومات مثل:

- Transaction ID
- Account number
- Transaction amount

- Date and time
- Transaction type

هذه البيانات تُستخدم في تحليل العمليات المشبوهة في الأنظمة المالية.

3. Method 1 – Pandas with Chunk Size

في هذه الطريقة استخدمنا مكتبة Pandas

لقراءة الملف على أجزاء صغيرة بدل تحميله بالكامل في الذاكرة

الفكرة

بدل قراءة الملف دفعة واحدة:

يتم تقسيمه إلى أجزاء (chunks) مثلا:

10000صف في كل مرة.

Example Code

```
import pandas as pd
import time

file = "HI-Large_Trans.csv"

start = time.time()

chunksize = 10000
rows = 0

for chunk in pd.read_csv(file,
chunksize=chunksize):
    rows += len(chunk)

end = time.time()

print("Total rows:", rows)
print("Execution time:", end - start)
```

Advantages

- تقليل استهلاك الذاكرة (RAM)
- مناسب للملفات الكبيرة

Result

(ضع الوقت الذي ظهر عندك) Execution Time:

4. Method 2 – Using Dask

في هذه الطريقة استخدمنا مكتبةDask

هذه المكتبة تعمل بطريقة مشابهة لـPandas

لكنها مصممة لمعالجة البيانات الضخمة باستخدام **المعالجة المتوازية**.

Example Code

```
import dask.dataframe as dd
import time

file = "HI-Large_Trans.csv"

start = time.time()

df = dd.read_csv(file)

rows = df.shape[0].compute()

end = time.time()
```

```
print("Total rows:", rows)
print("Execution time:", end - start)
```

Advantages

- أسرع مع البيانات الكبيرة
- يدعم المعالجة المتوازية
- مناسب لتطبيقاتBig Data

Result

Execution Time: (ضع الوقت الذي ظهر عندك)

---

5. Method 3 – Compression

في هذه الطريقة قمنا بضغط ملفCSV

باستخدام صيغة **gzip** لتقليل حجم التخزين

Example Code

```
import pandas as pd

file = "HI-Large_Trans.csv"

df = pd.read_csv(file)

df.to_csv("HI-Large_Trans.csv.gz",
compression="gzip", index=False)
```

بعد ذلك يمكن قراءة الملف المضغوط باستخدام:

```
df = pd.read_csv("HI-Large_Trans.csv.gz",
compression="gzip")
```

Advantages

- تقليل حجم التخزين
- تسهيل نقل البيانات

Result

Execution Time: (ضع الوقت الذي ظهر عندك)

---

6. Comparison

Method	Execution Time	Memory Usage	Advantage
Pandas Chunk	...	Low	مناسب للملفات الكبيرة
Dask	...	Medium	أسرع في معالجة البيانات
Compression	...	Low Storage	يقلل حجم الملف

---

7. Conclusion

في هذا المشروع قمنا بدراسة طرق مختلفة لمعالجة ملفاتCSV الكبيرة

النتائج أظهرت أن:

استخدام **Chunk Size في** Pandas

يسمح بمعالجة الملفات الكبيرة دون استهلاك كبير للذاكرة .

- مكتبة **Dask** توفر أداءً أفضل في معالجة البيانات الضخمة
- الضغط باستخدام **Compression** يساعد في تقليل حجم التخزين.

بالتالي، يعتمد اختيار الطريقة المناسبة على حجم البيانات والموارد المتاحة في النظام.